

Introduction

Linked list problems have a reputation for being either trivial or brutal, with not much in between. LeetCode 2058, "Find the Minimum and Maximum Number of Nodes Between Critical Points," lands right at the sweet spot interviewers love: the setup is easy to state, the naive solution is obvious, and the optimal solution requires you to notice something subtle about the structure of the problem. That's exactly the kind of question that separates candidates who can code from candidates who can reason about code.

What makes this problem worth your time isn't the linked list traversal itself — you've done that a hundred times. It's the realization, buried in the middle of the problem, that one of the two values you need to compute is *global* and the other is *local*. Spotting that distinction on the fly, under interview pressure, is the actual skill being tested. Once you've internalized it here, you'll start noticing the same shape in array problems, string problems, and anywhere else you're asked to track "closest" and "farthest" in the same pass.

Problem Overview

You're given the head of a singly linked list of integers. A node is called a **critical point** if it is either:

- A **local maximum**: strictly greater than both its previous and next neighbor, or
- A **local minimum**: strictly less than both its previous and next neighbor.

The head and tail of the list can never be critical points, because a critical point requires both a previous and a next neighbor to compare against.

Once you've identified every critical point in the list, you need to return two values:

1. **Minimum distance** — the smallest gap (in node count) between any two *consecutive* critical points.
2. **Maximum distance** — the gap between the very first and the very last critical point in the list.

If the list has fewer than two critical points, there's nothing to measure a distance between, so you return `[-1, -1]`.

Example

Take the list `[5, 3, 1, 2, 5, 1, 2]`, with indices `0` through `6`.

- Index `2` (value `1`): neighbors are `3` and `2`. Both are bigger, so this is a local minimum. Critical.
- Index `4` (value `5`): neighbors are `2` and `1`. Both are smaller, so this is a local maximum. Critical.
- Index `5` (value `1`): neighbors are `5` and `2`. Both are bigger, so this is a local minimum. Critical.

Critical point indices: `2, 4, 5`.

- **Minimum distance:** the closest consecutive pair is `4` and `5`, one node apart $\rightarrow 1$.
- **Maximum distance:** the outermost pair is `2` and `5` $\rightarrow 5 - 2 = 3$.

Result: `[1, 3]`.

Intuition

The first instinct most people have is to treat this like a generic "find all pairwise distances and take the min and max" problem. That instinct is half right and half a trap.

Think about what the maximum distance actually represents: the gap between the *first* critical point and the *last* critical point. No matter what critical points exist in between, none of them can ever be farther from the first point than the last point is, and none of them can ever be farther from the last point than the first point is. The two extremes of the list are, by definition, the two extremes of the distance. So the maximum distance doesn't need any tracking at all beyond remembering the first and last positions you've seen — it's `last - first`, full stop.

The minimum distance is the opposite. Two critical points that are far apart in the list tell you nothing about two critical points that happen to sit close together somewhere else. There's no shortcut here — you genuinely have to compare each critical point to the one immediately before it and keep the smallest gap you've found.

Once you see this asymmetry — one value is global and needs no running computation, the other is local and needs a running minimum — the algorithm falls out naturally. You don't need to store every critical point's index in a list. You only need to carry three numbers forward as you walk: the position of the first critical point, the position of the previous critical point, and the smallest gap seen so far. The last critical point's position doubles as what you need for the max distance at the end.

Brute Force Solution

Idea: Walk the list once to collect the index of every critical point into an array. Then do a second pass over that array, comparing each pair of neighbors to find the minimum gap, and subtract the first index from the last to get the maximum gap.

Algorithm: 1. Traverse the list, checking each node (except head and tail) against its neighbors. 2. Append the index of every critical point to a list, `positions`. 3. If `positions` has fewer than 2 entries, return `[-1, -1]`. 4. Compute `max_dist = positions[-1] - positions[0]`. 5. Loop through `positions`, tracking the smallest difference between consecutive entries as `min_dist`. 6. Return `[min_dist, max_dist]`.

Advantages: Straightforward to write, easy to reason about, low risk of an off-by-one bug because you can inspect the full `positions` array while debugging.

Disadvantages: Uses $O(k)$ extra space for the array of critical point positions, where k can be up to $n/2$ in the worst case. Unnecessary since you never need more than the first, previous, and last positions at once.

Complexity: $O(n)$ time (two linear passes), $O(n)$ space in the worst case for the positions array.

Optimal Solution

The optimal approach collapses the two-pass brute force into a single pass by only ever remembering three numbers instead of a full list of positions.

Set up two pointers, `prev` and `curr`, with `curr` one node ahead of `prev`, and an index counter `idx` starting at 1 (`curr`'s position). Before the loop starts, initialize:

Tracker	Purpose	Initial value
<code>first</code>	Index of the first critical point found	<code>None</code>
<code>prev_idx</code>	Index of the most recently found critical point	<code>None</code>
<code>last</code>	Index of the last critical point found (updated continuously)	<code>None</code>
<code>min_dist</code>	Smallest gap seen between consecutive critical points	<code>infinity</code>

Walk the list while `curr.next` exists:

1. Check if `curr` is a peak (`curr.val > prev.val` and `curr.val > curr.next.val`) or a valley (`curr.val < prev.val` and `curr.val < curr.next.val`).
2. If it is a critical point:
3. If `first` is still `None`, set `first = idx` — there's nothing to compare against yet.

4. Otherwise, compute `idx - prev_idx` and update `min_dist` if this gap is smaller.
5. Set `prev_idx = idx` and `last = idx` either way, so the next comparison uses the freshest critical point.
6. Advance `prev` and `curr` by one node each, and increment `idx`.

After the loop, if `first` is still `None` or `first == last` (meaning zero or one critical point was found), return `[-1, -1]`. Otherwise return `[min_dist, last - first]`.

Python Code

```
from typing import Optional, List

class ListNode:
    def __init__(self, val: int = 0, next: 'Optional[ListNode]' = None):
        self.val = val
        self.next = next

def nodes_between_critical_points(head: Optional[ListNode]) -> List[int]:
    prev = head
    curr = head.next
    idx = 1

    first = prev_idx = last = None
    min_dist = float('inf')

    while curr and curr.next:
        is_peak = curr.val > prev.val and curr.val > curr.next.val
        is_valley = curr.val < prev.val and curr.val < curr.next.val

        if is_peak or is_valley:
            if first is None:
                first = idx
            else:
                min_dist = min(min_dist, idx - prev_idx)
                prev_idx = idx
                last = idx

        prev = curr
        curr = curr.next
        idx += 1

    if first is None or first == last:
        return [-1, -1]

    return [min_dist, last - first]
```

Code Walkthrough

- `prev` and `curr` are the two pointers used to check a node against its immediate neighbors — `prev` trails one step behind `curr`.
- `idx` tracks `curr`'s position in the list, starting at `1` since `curr` begins at the second node (index `0` can never be critical anyway).
- `first`, `prev_idx`, and `last` all start as `None` because no critical point has been found yet; `min_dist` starts at infinity so the first real gap always replaces it.
- The loop condition `curr` and `curr.next` stops one node before the tail, since the tail can't be checked against a "next" neighbor.
- `is_peak` and `is_valley` are the two ways a node qualifies as critical — note the strict `>` and `<`, which excludes ties.
- The `if first is None` branch handles the very first critical point specially, since there's nothing yet to subtract from.
- Every subsequent critical point updates `min_dist` against the gap from the *previous* critical point only — never against `first` or any earlier point.
- `prev_idx` and `last` are updated together on every critical point hit; `last` simply always holds the most recent one, which by the end of the loop is the final one.
- After the loop, `first is None` catches zero critical points, and `first == last` catches exactly one (both trackers point at the same single index).

Dry Run

List: `[5, 3, 1, 2, 5, 1, 2]`

idx curr.val prev.val next.val Critical? first prev_idx last min_dist

1	3	5	1	no	—	—	—	inf
2	1	3	2	valley	2	2	2	inf
3	2	1	5	no	2	2	2	inf
4	5	2	1	peak	2	4	4	2 (4-2)
5	1	5	2	valley	2	5	5	1 (5-4)

Loop ends (`curr.next` is now the tail). `first = 2`, `last = 5`, both set, so we return `[min_dist, last - first] = [1, 3]` — matching the expected output.

Complexity Analysis

Time: $O(n)$. The list is walked exactly once, with a constant amount of work per node — no nested loops, no revisiting.

Space: $O(1)$. Regardless of list length, only a fixed handful of scalar variables (`first`, `prev_idx`, `last`, `min_dist`, `idx`, and the two pointers) are ever held in memory. Correctness holds because the maximum distance only ever needs the outermost two points (proven by the fact that nothing between them can be farther apart than they are), and the minimum distance only ever needs a comparison against the immediately preceding critical point — no historical data beyond that is relevant.

Alternative Solutions

The brute force two-pass approach (collect all positions, then scan for min/max) is the only reasonable alternative, and it's worth knowing because interviewers sometimes want you to state it first before optimizing. It trades $O(n)$ extra space for slightly more readable code — arguably useful if you're debugging by inspecting the full array of positions. But since the single-pass version is no harder to write once you understand the first/prev/last insight, there's rarely a reason to ship the brute force version in practice.

Edge Cases

- **Empty or near-empty list** (fewer than 3 nodes): no node can have both a previous and next neighbor, so there are zero critical points $\rightarrow [-1, -1]$.
- **Strictly monotonic list** (e.g., `[9, 8, 7, 6, 5]`): every interior node has one bigger and one smaller neighbor, so nothing qualifies as a peak or valley $\rightarrow [-1, -1]$.
- **Exactly one critical point:** `first == last`, nothing to pair it against $\rightarrow [-1, -1]$.
- **All values equal** (e.g., `[4, 4, 4, 4]`): no node is strictly greater or strictly smaller than both neighbors, so no critical points exist.
- **Critical points at the very ends of the "interior"** (index 1 or $n-2$): still valid as long as they have both a previous and next node — only the true head and tail are excluded.

Common Mistakes

1. **Allowing ties to count as critical.** The problem requires *strict* inequality on both sides — a node equal to a neighbor is neither a peak nor a valley.
2. **Computing max distance as the largest gap between *any* pair of critical points** instead of just `last - first`. This is unnecessary extra work and, if implemented incorrectly,

can actually give the wrong answer for edge orderings.

3. **Forgetting to guard against fewer than two critical points**, causing a crash (subtracting from `None`) or a silently wrong answer instead of returning `[-1, -1]`.
4. **Checking the head or tail node as if they could be critical**. They can't — they lack a full pair of neighbors — and checking them either crashes on a null neighbor access or produces false positives.
5. **Off-by-one errors in index tracking**, especially when the pointer-based single-pass approach is used and `idx` isn't incremented in sync with `curr`.

Interview Questions

- How would this change if the list were doubly linked instead of singly linked?
- Can you solve this without extra pointers, using recursion instead? What are the tradeoffs?
- What if the list were circular — would the head and tail be able to become critical points?
- How would you extend this to return the actual node values at the min/max distance positions, not just indices?
- Could this same first/previous/last tracking pattern apply to a streaming input where you can't look ahead?

Similar Problems

- **LeetCode 852 (Peak Index in a Mountain Array)** — same core idea of identifying a local maximum, but on an array with binary search instead of linear scan.
- **LeetCode 1901 (Find a Peak Element II)** — extends peak-finding to two dimensions.
- **LeetCode 876 (Middle of the Linked List)** — good companion practice for the two-pointer technique used here.
- **LeetCode 2130 (Maximum Twin Sum of a Linked List)** — another linked list problem that rewards tracking positions cleverly instead of storing the whole list.
- **LeetCode 162 (Find Peak Element)** — the array analog of peak detection, useful for contrasting linked list vs. array traversal tradeoffs.

Key Takeaways

A critical point is a strict local peak or valley, and only interior nodes qualify. The maximum distance between critical points is always just the gap between the first and last one found — never a value that needs a running comparison. The minimum distance, by contrast, is inherently local and requires

comparing each critical point only to the one immediately before it. Recognizing that asymmetry is what turns a two-pass, extra-space solution into a clean single pass with constant space.

Watch the Video

If you'd rather see this traced live, node by node, with the two in-video quizzes to test whether the pattern actually stuck, watch the full walkthrough here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a daily series that takes one coding interview problem and works through it from first principles — building the intuition before ever touching code, then proving out the optimal solution step by step. If you're prepping for interviews or just enjoy watching problems get taken apart methodically, this series is built for you.

Call To Action

If this explanation helped the pattern click, give the video a like on YouTube and subscribe for daily problem breakdowns. For the full written version delivered straight to your inbox, subscribe to the Substack newsletter. Drop a comment with the problem you want tackled next, and share this with anyone grinding through their own interview prep.

Quick Review

Pattern tell: linked list where each value compared to prev AND next?

Local min/max scan — track prev, curr, next in one pass; that's the 'critical point' signal.

Time complexity for scanning the list for critical points?

$O(n)$ — single traversal, comparing each node's neighbors as you go.

Space complexity for min/max distance between critical points?

$O(1)$ — only track first index, previous index, and running min/max, no storage of the list.

Trickiest edge case with critical points?

Head and tail nodes can't be critical (no left or right neighbor) — start comparisons from the second node.

Min distance vs max distance between critical points — different logic?

Max = last minus first (one pass, no extra state); min = smallest gap between *consecutive* critical points only.

Follow-up: why can't you just store all critical point indices in a list first?

You can, but it's wasted $O(k)$ space — tracking first/prev index while streaming gets both answers in $O(1)$ space.

Find the Minimum and Maximum Number of Nodes Between Critical Points — free Python cheat sheet